

Sample Knowledge Base for Local RAG Testing

A synthetic multi-section PDF designed for testing PDF extraction, chunking, dense retrieval, BM25 retrieval, hybrid ranking, and local LLM generation.

Topic	Purpose
RAG	Retrieval + generation architecture
OpenSearch	Lexical and vector retrieval backend
BM25	Exact-term lexical ranking
Embeddings	Semantic representation
Hybrid Search	Combine semantic and lexical rankings
Ollama	Local language-model generation

1. Retrieval-Augmented Generation Fundamentals

Retrieval-Augmented Generation, commonly called RAG, combines information retrieval with language-model generation. Instead of asking a model to answer only from parameters learned during training, a RAG system first searches a knowledge base for relevant passages and then provides those passages as context to the model.

A typical RAG pipeline contains ingestion, chunking, embedding, indexing, retrieval, prompt construction, generation, and evaluation. The ingestion phase reads source documents. Chunking divides long documents into smaller units. Embeddings convert chunks into numeric vectors. A search system stores both text and vectors. At question time, the retriever selects useful chunks and a language model produces a grounded answer from them.

RAG is useful when knowledge changes frequently, when source documents are private, or when answers need to be traceable to supplied material. It does not guarantee correctness by itself. Poor chunking, weak retrieval, stale documents, and prompts that ignore evidence can all reduce answer quality.

Practical note: In a real system, retrieval quality should be measured with representative questions and known relevant passages rather than judged only from one demo query.

2. Text Embeddings and Semantic Similarity

An embedding is a dense numeric representation of text. Modern sentence-embedding models map semantically similar passages to nearby locations in vector space. For example, the phrases "car repair instructions" and "how to fix an automobile" may have high similarity even though they share few exact words.

Sentence Transformers are widely used for local embedding generation. A model such as all-MiniLM-L6-v2 produces a fixed-size vector for each input sentence or chunk. In one common configuration the vector has 384 dimensions. The dimension is the number of numeric coordinates in each embedding; it is not the number of search results.

Cosine similarity measures the angle between vectors and is frequently used to compare embeddings. Vector databases and search engines can use approximate nearest-neighbor algorithms to find the closest vectors efficiently, even when millions of chunks have been indexed.

3. OpenSearch as a Retrieval Backend

OpenSearch is a distributed search and analytics engine. It can store full text, metadata, and vector fields in the same index. This makes it a useful backend for hybrid RAG systems because one engine can execute both lexical search and semantic vector search.

An OpenSearch index is a persistent searchable collection of documents. Each document contains fields defined by a mapping. A RAG index might contain a text field, a document_name field, a chunk_number field, and an embedding field. The text field can be analyzed for BM25 retrieval while the embedding field can be configured as a k-NN vector.

The Python opensearch-py client is an application-side interface that sends HTTP requests to the OpenSearch server. The client is not the server and it is not an index. One client can create, update, search, and delete many indexes on the same cluster.

4. BM25 Lexical Search

BM25 is a classic ranking function for lexical search. It rewards documents that contain important query terms while accounting for term frequency, document length, and how rare each term is across the collection. It is especially strong when the user query contains exact names, identifiers, error messages, product codes, or domain-specific terminology.

In OpenSearch, a match query against a text field can use BM25 scoring by default. A query such as "Python programming language" is likely to rank a chunk containing those exact terms highly. BM25 may return fewer than the requested result size when only a small number of indexed documents match the query terms.

Lexical search complements semantic search. Semantic retrieval can recognize meaning expressed with different words, while BM25 can preserve exact keyword precision. Hybrid systems combine the strengths of both approaches.

Practical note: In a real system, retrieval quality should be measured with representative questions and known relevant passages rather than judged only from one demo query.

5. Dense Vector Search and k-NN

Dense retrieval converts the user query into an embedding and searches for nearby document embeddings. A k-nearest-neighbor query asks the vector index for the k closest candidates. If k is three and the index contains at least three eligible documents, the search can return three nearest vectors even when some are only weakly related.

Exact nearest-neighbor search becomes expensive as collections grow. Approximate nearest-neighbor methods trade a small amount of recall for much faster retrieval. HNSW, short for Hierarchical Navigable Small World graphs, is a common ANN structure that links vectors in a graph and traverses that graph to find nearby candidates efficiently.

k is the number of neighbors requested. Vector dimension is the number of values in each embedding. These concepts are independent. A system may use 384-dimensional embeddings and request the top 5 nearest chunks.

6. Hybrid Retrieval and Rank Fusion

Hybrid retrieval combines lexical and semantic search. The main challenge is that BM25 scores and vector-similarity scores are not necessarily comparable. A BM25 score of 3.1 and a vector score of 0.72 come from different scoring systems, so directly adding them can be misleading.

Rank fusion avoids comparing raw scores by using result positions. A simple reciprocal-rank scheme gives higher weight to documents near the top of each list. For example, rank one can contribute 1.0, rank two 0.5, and rank three about 0.33. Documents appearing highly in both lexical and semantic lists accumulate a larger combined score.

Classic Reciprocal Rank Fusion often uses a formula such as $1 / (k + \text{rank})$, where k is a smoothing constant. Production systems may also normalize scores, use learned weights, or rerank candidates with a cross-encoder.

Practical note: In a real system, retrieval quality should be measured with representative questions and known relevant passages rather than judged only from one demo query.

7. Document Chunking

Large documents should usually be split before embedding. If an entire PDF is embedded as one vector, retrieval can identify only the document as a whole and may miss the specific paragraph needed to answer a question. Smaller chunks provide more precise retrieval units.

A simple word-based chunker can split extracted PDF text into a list of words, take a fixed number of words for each chunk, and move forward with an overlap. For example, a chunk size of 500 words and an overlap of 100 words produces starts at approximately 0, 400, 800, and so on.

Overlap protects context near boundaries. If an important explanation begins near the end of one chunk, the overlapping region allows part of that explanation to appear in the next chunk too. More advanced splitters attempt to preserve paragraph, sentence, or section boundaries.

Practical note: In a real system, retrieval quality should be measured with representative questions and known relevant passages rather than judged only from one demo query.

8. PDF Text Extraction

A PDF is a visual document format, so text extraction is not always as clean as reading a plain text file. Libraries such as `pypdf` can extract text from many digitally generated PDFs. Each page can be processed independently with an `extract_text` call and the page strings can then be combined.

Scanned PDFs may contain images of text rather than embedded text. In that case, normal extraction can return little or nothing and optical character recognition is needed. OCR tools such as Tesseract can convert page images into machine-readable text, although OCR errors may need cleanup.

For a learning project, starting with normal text-based PDFs is useful because it isolates retrieval logic from OCR complexity. OCR can be added later as a fallback for pages where normal extraction returns no useful text.

9. Local LLM Generation with Ollama

Ollama provides a convenient way to run supported language models locally. In a local RAG architecture, OpenSearch retrieves the relevant chunks and Ollama receives those chunks together with the user question. The language model then generates an answer using the supplied context.

Ollama does not replace the retriever. The retriever decides which source text is relevant. The language model is responsible for generation. Keeping these responsibilities separate makes the architecture easier to understand, test, and debug.

Local generation can be attractive for privacy-sensitive workflows because documents do not need to be sent to a hosted model provider. The practical tradeoffs are local hardware requirements, model size, inference speed, and model quality.

Practical note: In a real system, retrieval quality should be measured with representative questions and known relevant passages rather than judged only from one demo query.

10. Docker and Local Services

Docker packages software and its dependencies into containers. For this project, OpenSearch can run in a container while the Python application runs in a local virtual environment. A Docker port mapping such as 9200:9200 forwards requests from port 9200 on the host machine to port 9200 inside the container.

Running curl against `http://localhost:9200` is a useful health check because it tests the OpenSearch HTTP server independently of the Python code. The Python client later communicates with that same service programmatically.

Containers are isolated process environments rather than full virtual machines. They make complex services easier to start consistently across development machines and reduce the amount of software that must be installed directly on the host operating system.

11. Deterministic Document IDs

When documents are indexed without an explicit ID, OpenSearch can generate a new identifier each time. Re-running an ingestion script may therefore create duplicates. A deterministic ID prevents this by producing the same ID for the same logical chunk.

A SHA-256 hash of chunk text is a simple learning-project strategy. The same text produces the same hash, so indexing again updates or replaces the existing document instead of creating another copy. In larger systems, IDs often incorporate source document identity, page number, chunk number, or a stable external record key.

Deterministic IDs make ingestion idempotent: repeating the same ingestion operation has a predictable result. This is useful in development and production pipelines that may be retried.

12. Evaluation and Debugging

RAG systems should be tested at each layer rather than only through the final chatbot. First verify that text extraction works. Then inspect chunks. Next verify embeddings and index counts. Test semantic retrieval with paraphrased queries and BM25 with exact-keyword queries. Finally inspect hybrid rankings before adding the language model.

Logging is preferable to scattered print statements in a maintainable application. Logs can show index creation, ingestion counts, query latency, retrieved document IDs, and final rankings. Avoid logging giant embedding vectors unless debugging a specific vector problem.

A useful debugging principle is to inspect the structure returned by an API before writing deep dictionary access code. For example, OpenSearch search responses contain an outer hits object and an inner hits list. Understanding that structure makes expressions such as `response["hits"]["hits"][0]["_source"]["text"]` straightforward rather than magical.